

ENTREGA PARCIAL 3 · DOCUMENTO 2

Descripción de la persistencia

Parkings Together — Modelo de datos, seguridad y lógica en PostgreSQL

1. Motor y estrategia de persistencia

La persistencia se implementa sobre Supabase, una plataforma gestionada cuyo núcleo es PostgreSQL. El acceso desde la aplicación se realiza con la librería oficial supabase-js, encapsulada en el paquete compartido packages/supabase-db. Se emplean dos clientes: uno anónimo para las lecturas públicas, y uno construido con el JWT del usuario (getSupabaseWithToken) para las escrituras, de modo que las políticas de seguridad de la base de datos evalúen la identidad real del solicitante.

Recurso de persistencia (rúbrica): el proyecto utiliza procedimientos almacenados (stored procedures) en PL/pgSQL como recurso de persistencia transaccional. Al no estar implementado en Java, no se emplea JPA; los procedimientos almacenados son la alternativa equivalente y válida, y concentran la integridad en el propio motor de base de datos.

2. Modelo de datos

El esquema se compone de cuatro tablas en el esquema public, todas con Row Level Security habilitada. La tabla perfiles extiende a los usuarios de Supabase Auth (auth.users).

2.1. Tabla perfiles

Columna	Descripción
id (uuid, PK)	Referencia a auth.users (1:1). Clave primaria.
nombre (text)	Nombre del usuario. Obligatorio.
telefono (text)	Teléfono de contacto (opcional).
avatar_url (text)	URL de la foto de perfil (opcional).
requiere_pmr (bool)	Si requiere plaza para personas con movilidad reducida.
rol (text)	'cliente' o 'anfitrión' (CHECK).
created_at (timestampz)	Fecha de creación (UTC).

2.2. Tabla vehiculos

Columna	Descripción
id (uuid, PK)	Identificador del vehículo.
user_id (uuid, FK)	Dueño del vehículo (auth.users).
placa (text)	Patente. Obligatoria.
modelo (text)	Modelo del vehículo.
color (text)	Color del vehículo.
created_at (timestampz)	Fecha de creación (UTC).

2.3. Tabla estacionamientos

Columna	Descripción
id (uuid, PK)	Identificador del estacionamiento.
user_id (uuid, FK)	Anfitrión propietario (auth.users).
nombre (text)	Nombre del estacionamiento.
arrendador (text)	Nombre del arrendador.
lat / lng (numeric)	Coordenadas geográficas.
total_spots (int)	Capacidad total de plazas.
occupied_spots (int)	Plazas ocupadas. CHECK: $0 \leq \text{occupied} \leq \text{total}$.
es_pmr (bool)	Plaza accesible (PMR).
precio_hora (int)	Precio por hora en CLP (regla de negocio).
created_at (timestampz)	Fecha de creación (UTC).

2.4. Tabla reservas

Columna	Descripción
id (uuid, PK)	Identificador de la reserva.
estacionamiento_id (FK)	Estacionamiento reservado (ON DELETE CASCADE).
conductor_id (uuid, FK)	Conductor que reserva (auth.users).
estado (text)	'activa', 'completada' o 'cancelada' (CHECK).
created_at (timestampz)	Fecha de creación (UTC).

3. Relaciones e integridad referencial

perfiles mantiene una relación 1:1 con `auth.users`. Un usuario (anfitrión) posee N estacionamientos y N vehículos. Un estacionamiento recibe N reservas, y un conductor realiza N reservas. Las claves foráneas usan `ON DELETE CASCADE` donde corresponde, garantizando que no queden registros huérfanos. Un `CHECK` en estacionamientos asegura que la ocupación nunca sea negativa ni supere la capacidad total.

4. Seguridad: Row Level Security (RLS)

Todas las tablas tienen RLS habilitada; el acceso se rige por políticas evaluadas contra `auth.uid()`:

- **perfiles y vehiculos:** cada usuario solo puede ver y modificar sus propios registros (`auth.uid() = id / user_id`).
- **estacionamientos:** lectura pública (`USING true`); solo usuarios con rol 'anfitrión' pueden crear, y cada anfitrión solo actualiza o elimina los suyos.
- **reservas:** el conductor ve y gestiona sus reservas; además, el anfitrión puede ver las reservas de sus estacionamientos.

5. Lógica de negocio en la base de datos (procedimientos almacenados)

Las operaciones transaccionales críticas se implementan como funciones PL/pgSQL con `SECURITY DEFINER`, lo que garantiza atomicidad y autorización del lado del servidor de base de datos:

5.1. `reservar_estacionamiento(p_estacionamiento_id uuid)`

Realiza la reserva de forma atómica: bloquea la fila del estacionamiento (`SELECT ... FOR UPDATE`) para evitar la doble reserva concurrente, valida que exista sesión (`auth.uid()`) y que haya cupo disponible, inserta la reserva del conductor y aumenta la ocupación en una única transacción. Si el estacionamiento está lleno, lanza una excepción que la API traduce a un código HTTP 409. Solo se concede `EXECUTE` al rol `authenticated`.

5.2. `cancelar_reserva(p_reserva_id uuid)`

Cancela una reserva activa y libera el cupo (decrementa la ocupación con un mínimo de 0). Valida que quien cancela sea el conductor dueño de la reserva y que esta se encuentre en estado 'activa'.

5.3. Trigger `handle_new_user()`

Trigger AFTER INSERT sobre `auth.users` que crea automáticamente el perfil del usuario al registrarse, tomando el nombre y el rol de los metadatos. Es idempotente (ON CONFLICT DO NOTHING) y resuelve el caso en que la confirmación por correo está activada.

6. Conclusión

La estrategia de persistencia concentra la integridad y la seguridad en PostgreSQL: el modelo relacional con claves foráneas y restricciones CHECK garantiza la consistencia de los datos; las políticas RLS aseguran el acceso por usuario; y los procedimientos almacenados encapsulan las transacciones críticas (reserva y cancelación) de forma atómica y resistente a la concurrencia. Este enfoque cumple el requisito de la rúbrica de contar con un recurso de persistencia robusto mediante procedimientos almacenados.